[Euron] 머신러닝 스터디 6주

통계 수정 삭제

조윤서 · 방금 전

0

(Euron) 머신러닝

목록 보기

5/5

교재: 파이썬 머신러닝 완벽 가이드 (권철민)

제 5장. 회귀

01. 회귀 소개

- ***회귀***: 여러 개의 독립변수와 한 개의 종속변수 간의 상관관계를 모델링하고 예측

- 데이터 값이 평균과 같은 일정한 값으로 돌아가려는 경향을 이용한 통계학 기법
- 엔지니어링, 의학, 사회과학, 경제학 등의 분야에 기여
- 갈톤(Galton)이 수행한 연구에서 유래
- 독립변수-피처, 종속변수-결정 값

- **머신러닝 회귀 예측의 핵심**: 주어진 피처와 결정 값 데이터 기반에서 학습을 통해 최적의 회귀 계수를 찾아내는 것

- **회귀 유형**

유형	특징
단일회귀	독립변수가 1개

유형	특징
다중 회귀	독립변수가 여러 개
선형 회귀	회귀 계수가 선형으로 결합
비선형 회귀	회귀 계수가 비선형으로 결합

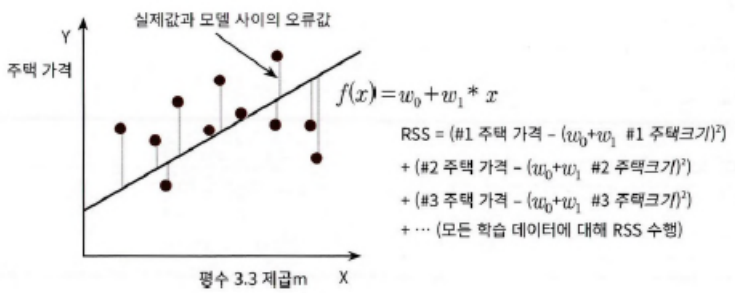
- **선형 회귀**: 실제 값과 예측값의 차이(오류의 제곱 값)를 최소화하는 직선형 회귀선을 최적화하는 방식

선형 회귀 유형	적용 규제	특징
일반 선형 회귀	X	예측값과 실제 값의(RSS)를 최소화할 수 있도록 회귀 계수를 최적화
릿지(Ridge) 회귀	L2 규제	L2규제: 상대적으로 큰 회귀 계수 값의 예측 영향도를 감소시키기 위해 회귀 계수값을 더 작게 만듦
라쏘(Lasso) 회귀	L1 규제	L1 규제(= 피처 선택 기능): 피처와 회귀 계수를 0으로 만들어 회귀 예측 시 피처가 선택되지 않게 함
엘라스틱넷	L2, L1 규제 결합	주로 피처가 많은 데이터 세트에 적용되며, L1 규제로 피처의 개수를 줄임과 동시에 L2 규제로 계수 값의 크기를 조정
로지스틱 회귀	-	강력한 분류 알고리즘, 이진 분류뿐만 아니라 희소 영역의 분류(ex.텍스트 분류)에서 뛰어난 예측 성능을 보임

02. 단순 선형 회귀를 통한 회귀 이해

- **단순 선형 회귀**: 독립변수와 종속변수가 각각 1개인 선형 회귀
- **잔차**(=남은 오류): 실제 값과 회귀 모델의 차이에 따른 오류 값

- RSS(Residual Sum of Square): 오류 값의 제곱을 구해서 더하는 방식



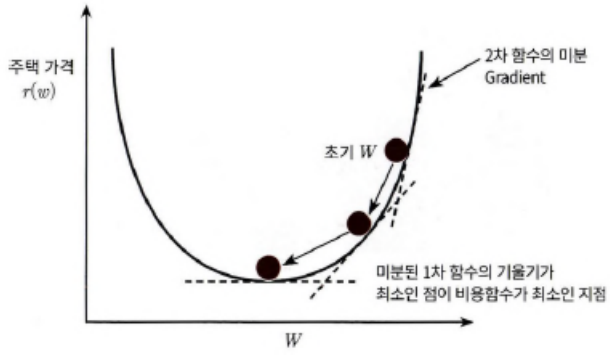
$$RSS(w_0, w_1) = \frac{1}{N}\sum_{i=1}^{N}(y_i - (w_0 + w_1 * x_i))^2$$

(*i*는 1부터 학습 데이터의 총 건수 *N*까지)

- RSS를 최소로하는 *회귀 계수*를 학습을 통해 찾는 것이 머신러닝 기반 회귀이 핵심
- 비용 함수(RSS)가 반환하는 값(즉, 오류 값)을 지속해서 감소시키고 최종적으로는 더 이상 감소하지 않는 최소의 오류를 구함
- 비용 함수 = 손실 함수(loss function)

03. 비용 최소화하기 - 경사 하강법(Gradient Descent) 소개

- **경사 하강법**: 점진적으로 반복적인 계산을 통해 W 파라미터 값을 업데이트하면서 오류 값이 최소가 되는 W 파라미터를 구하는 방식



$$\frac{\partial R(w)}{\partial w_1} = \frac{2}{N}\sum_{i=1}^{N} -x_i * (y_i - (w_0 + w_1 x_i)) = -\frac{2}{N}\sum_{i=1}^{N} x_i * (\text{실제값}_i - \text{예측값}_i)$$

$$\frac{\partial R(w)}{\partial w_0} = \frac{2}{N}\sum_{i=1}^{N} -(y_i - (w_0 + w_1 x_i)) = -\frac{2}{N}\sum_{i=1}^{N} (\text{실제값}_i - \text{예측값}_i)$$

- Step 1: w_1, w_0를 임의의 값으로 설정하고 첫 비용 함수의 값을 계산합니다.
- Step 2: w_1을 $w_1 + \eta \frac{2}{N}\sum_{i=1}^{N} x_i * (\text{실제값}_i - \text{예측값}_i)$, w_0을 $w_0 + \eta \frac{2}{N}\sum_{i=1}^{N} (\text{실제값}_i - \text{예측값}_i)$으로 업데이트한 후 다시 비용 함수의 값을 계산합니다.
- Step 3: 비용 함수가 감소하는 방향성으로 주어진 횟수만큼 Step 2를 반복하면서 w_1과 w_0를 계속 업데이트합니다.

파이썬 코드로 경사 하강법 구현하기

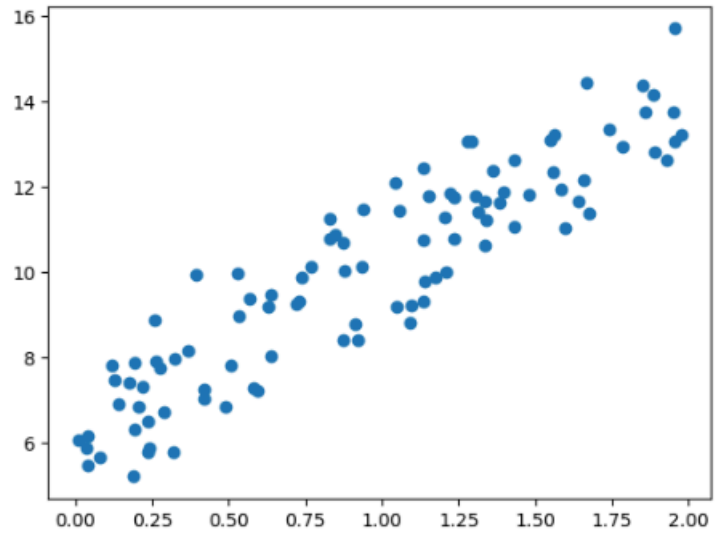
y = 4X + 6을 근사(w1=4, w0=6) + 노이즈

```
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

np.random.seed(0)
X = 2 * np.random.rand(100, 1)
y = 6 + 4*X + np.random.randn(100, 1)

plt.scatter(X, y)
```

```
<matplotlib.collections.PathCollection at 0x7b9a1b58a120>
```



- 100개의 데이터 세트
- 데이터는 y = 4X + 6 중심으로 무작위로 퍼져 있음

비용 함수 정의

```
def get_cost(y, y_pred):
  N = len(y)
  cost = np.sum(np.square(y - y_pred))/N
  return cost
```

- 실제 y값과 예측된 y값을 인자로 받아 계산

가중치(w1)와 절편(w0)의 수정량을 계산하는 함수 정의

```
def get_weight_updates(w1, w0, X, y, learning_rate=0.01):
  N = len(y)

  w1_update = np.zeros_like(w1)
  w0_update = np.zeros_like(w0)

  y_pred = np.dot(X, w1.T) + w0
  diff = y - y_pred

  w0_factors=np.ones((N, 1))

  w1_update = -(2/N)*learning_rate*(np.dot(X.T, diff))
  w0_update = -(2/N)*learning_rate*(np.dot(w0_factors.T, diff))

  return w1_update, w0_update
```

- 예측값과 실제값의 오차를 구한 뒤, 학습률을 적용해 가중치를 얼마나 조정할지 계산

가중치(w1)와 절편(w0)을 업데이트하는 함수 정의

```
def gradient_descent_steps(X, y, iters=10000):
  w0 = np.zeros((1, 1))
  w1 = np.zeros((1, 1))

  for ind in range(iters):
    w1_update, w0_update = get_weight_updates(w1, w0, X, y, learning_rate=0.01)
    w1 = w1 - w1_update
    w0 = w0 - w0_update

  return w1, w0
```

- 입력 인자 iters로 주어진 횟수만큼 반복적으로 w1과 w0를 업데이트

RSS를 계산하는 함수 정의

```
def get_cost(y, y_pred):
  N = len(y)
  cost = np.sum(np.square(y - y_pred))/N
  return cost
```

경사 하강법 예측 오류 계산

```
w1, w0 = gradient_descent_steps(X, y, iters=1000)
print('w1: {:.3f}, w0: {:.3f}'.format(w1[0, 0], w0[0,0]))
y_pred = w1[0,0]*X + w0[0,0]
print('Gradient Descent Total Cost: {:.4f}'.format(get_cost(y, y_pred)))
```

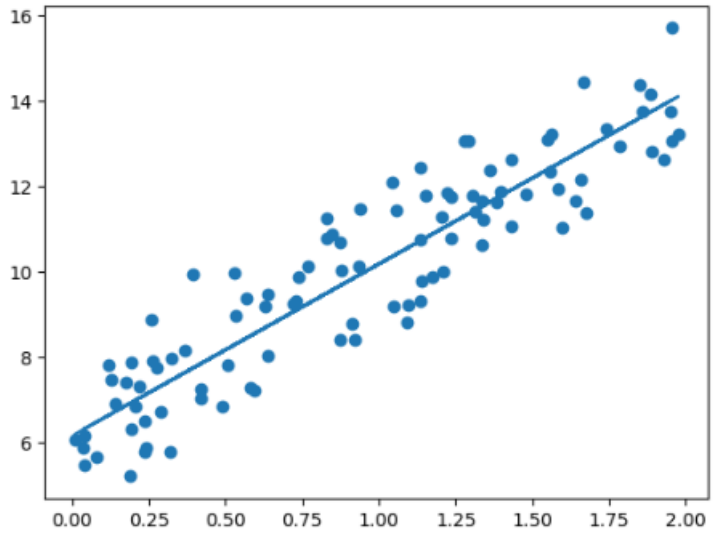
```
w1: 4.022, w0: 6.162
Gradient Descent Total Cost: 0.9935
```

- w1은 4.022, w0은 6.162
- 예측 오류 비용: 약 0.9935

회귀선

```
plt.scatter(X, y)
plt.plot(X, y_pred)
```

```
[<matplotlib.lines.Line2D at 0x7b9a181a39d0>]
```



확률적 경사 하강법(Stochastic Gradient Descent)

- 경사 하강법은 모든 학습 데이터에 대해 반복적으로 비용함수 최소화를 위한 값을 업데이트하기 때문에 수행 시간이 매우 오래 걸림 -> 확률적 경사 하강법

- **확률적 경사 하강법**: 일부 데이터만 이용해 w가 업데이트 되는 값을 계산

- 경사 하강법에 비해 빠른 속도

(미니 배치) 확률적 경사 하강법 구현 함수

```
def stochastic_gradient_descent_steps(X, y, batch_size=10, iters=1000):
  w0 = np.zeros((1, 1))
  w1 = np.zeros((1, 1))

  for ind in range(iters):
    np.random.seed(ind)

    stochastic_random_index = np.random.permutation(X.shape[0])
    sample_X = X[stochastic_random_index[0:batch_size]]
    sample_y = y[stochastic_random_index[0:batch_size]]

    w1_update, w0_update = get_weight_updates(w1, w0, sample_X, sample_y, learning_rate=0.01)
    w1 = w1 - w1_update
    w0 = w0 - w0_update

  return w1, w0
```

- 전체 X, y 데이터에서 랜덤하게 `batch_size` 만큼 데이터를 추출해 이를 기반으로 w1_update와 w0_update를 계산

w1, w0, 예측 오류 비용 계산

```
w1, w0 = stochastic_gradient_descent_steps(X, y, iters=1000)
print('w1:', round(w1[0,0], 3), 'w0:', round(w0[0,0], 3))
y_pred = w1[0, 0]*X + w0
print('Stochastic Gradient Descent Total Cost: {:.4f}'.format(get_cost(y, y_pred)))
```

```
w1: 4.028 w0: 6.156
Stochastic Gradient Descent Total Cost: 0.9937
```

- w1은 4.028, w0은 6.156
- 예측 오류 비용: 약 0.9937

- 피처가 M개 있다면 그에 따른 회귀 계수도 M+1개 도출

$$\hat{Y} \qquad X_{mat}$$

$$\begin{matrix} & \text{Feature 1} & \text{Feature 2} & \cdots & \text{Feature M} \end{matrix}$$

$$\begin{bmatrix} y_1 \\ y_2 \\ \cdots \\ y_n \end{bmatrix} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1m} \\ x_{21} & x_{22} & \cdots & x_{2m} \\ \cdots & \cdots & \cdots & \cdots \\ x_{n1} & x_{n2} & \cdots & x_{nm} \end{bmatrix} \underset{\text{내적}}{*} \begin{bmatrix} w_1 & w_2 & \cdots & w_m \end{bmatrix}^T + w_0$$

w_0를 Weight의 배열인 W안에 포함시키기 위해서 Xmat의 맨 처음 열에 모든 데이터의 값이 1인 피처 Feat 0을 추가하겠습니다. 이제 회귀 예측값은 $\hat{Y} = X_{mat} * W^T$와 같이 도출할 수 있습니다.

$$\hat{Y} \quad \text{1값을 가진 피처 추가} \quad X_{mat}$$

$$\begin{matrix} & \text{Feat 0} & \text{Feat 1} & \text{Feat 2} & \cdots & \text{Feat M} \end{matrix}$$

$$\begin{bmatrix} y_1 \\ y_2 \\ \cdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1m} \\ 1 & x_{21} & x_{22} & \cdots & x_{2m} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{nm} \end{bmatrix} \underset{\text{내적}}{*} \begin{bmatrix} w_0 & w_1 & w_2 & \cdots & w_m \end{bmatrix}^T \quad (w_0 \text{을 } W \text{ 배열 내에 포함})$$

$$\hat{Y} = X_{mat} * W^T$$

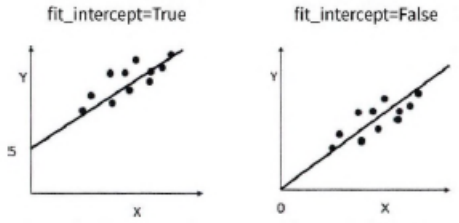
04. 사이킷런 LinearRegression을 이용한 보스턴 주택 가격 예측

사이킷런의 `linear_models` 모듈은 매우 다양한 종류의 선형 기반 회귀를 클래스로 구현해 제공

LinearRegression 클래스 - Ordinary Least Squares

- `LinearRegression` 클래스는 예측값과 실제 값의 RSS를 최소화해 OLS(Ordinary Least Squares) 추정 방식으로 구현한 클래스

```
class sklearn.linear_model.LinearRegression(fit_intercept=True, normalize=
```

입력 파라미터	**fit_intercept**: 불린 값으로, 디폴트는 True입니다. Intercept(절편) 값을 계산할 것인지 말지를 지정합니다. 만일 False로 지정하면 intercept가 사용되지 않고 0으로 지정됩니다.  **normalize**: 불린 값으로 디폴트는 False입니다. fit_intercept가 False인 경우에는 이 파라미터가 무시됩니다. 만일 True이면 회귀를 수행하기 전에 입력 데이터 세트를 정규화합니다.
속성	**coef_**: fit() 메서드를 수행했을 때 회귀 계수가 배열 형태로 저장하는 속성. Shape는 (Target 값 개수, 피처 개수). **intercept_**: intercept 값

회귀 평가 지표

- 실제 값과 회귀 예측값의 *차이 값*을 기반으로 한 지표가 중심

평가 지표	설명	수식
MAE	Mean Absolute Error(MAE)이며 실제 값과 예측값의 차이를 절댓값으로 변환해 평균한 것입니다.	$MAE = \frac{1}{n}\sum_{i=1}^{n}\lvert Yi - \hat{Y}i\rvert$
MSE	Mean Squared Error(MSE)이며 실제 값과 예측값의 차이를 제곱해 평균한 것입니다.	$MSE = \frac{1}{n}\sum_{i=1}^{n}(Yi - \hat{Y}i)^2$
RMSE	MSE 값은 오류의 제곱을 구하므로 실제 오류 평균보다 더 커지는 특성이 있으므로 MSE에 루트를 씌운 것이 RMSE(Root Mean Squared Error)입니다.	$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(Yi - \hat{Y}i)^2}$
R^2	분산 기반으로 예측 성능을 평가합니다. 실제 값의 분산 대비 예측값의 분산 비율을 지표로 하며, 1에 가까울수록 예측 정확도가 높습니다.	$R^2 = \frac{\text{예측값 } Variance}{\text{실제값 } Variance}$

평가 방법	사이킷런 평가 지표 API	Scoring 함수 적용 값
MAE	metrics.mean_absolute_error	'neg_mean_absolute_error'
MSE	metrics.mean_squared_error	'neg_mean_squared_error'
RMSE	metrics.mean_squared_error를 그대로 사용하되 squared 파라미터를 False로 설정.	'neg_root_mean_squared_error'
MSLE	metrics.mean_squared_log_error	'neg_mean_squared_log_error'
R^2	metrics.r2_score	'r2'

LinearRegression을 이용해 보스턴 주택 가격 회귀 구현

보스턴 주택 가격 데이터 피처



- CRIM: 지역별 범죄 발생률
- ZN: 25,000평방피트를 초과하는 거주 지역의 비율
- INDUS: 비상업 지역 넓이 비율
- CHAS: 찰스강에 대한 더미 변수(강의 경계에 위치한 경우는 1, 아니면 0)
- NOX: 일산화질소 농도
- RM: 거주할 수 있는 방 개수
- AGE: 1940년 이전에 건축된 소유 주택의 비율
- DIS: 5개 주요 고용센터까지의 가중 거리
- RAD: 고속도로 접근 용이도
- TAX: 10,000달러당 재산세율
- PTRATIO: 지역의 교사와 학생 수 비율
- B: 지역의 흑인 거주 비율
- LSTAT: 하위 계층의 비율
- MEDV: 본인 소유의 주택 가격(중앙값)

(1) 데이터 불러오기

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from scipy import stats
import warnings
warnings.filterwarnings('ignore')
%matplotlib inline
```

```
columns = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE',
           'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT', 'PRICE']

bostonDF = pd.read_csv('housing.csv', sep=r'\s+', header=None,
                       names=columns)


print('Boston 데이터 세트 크기:', bostonDF.shape)
bostonDF.head()
```

```
Boston 데이터 세트 크기: (506, 14)
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	PRICE
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1.0	296.0	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2.0	242.0	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2.0	242.0	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3.0	222.0	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3.0	222.0	18.7	396.90	5.33	36.2

```
bostonDF.info()
```

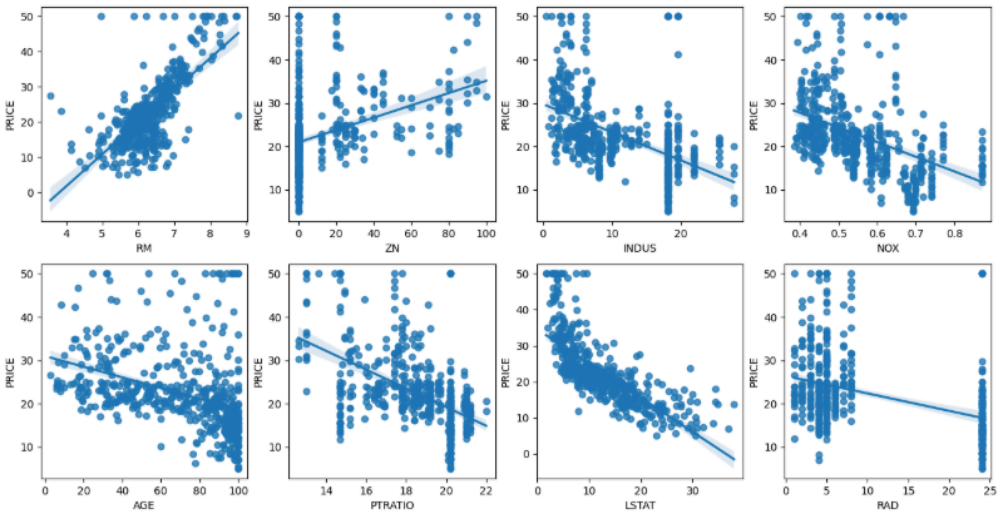
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   CRIM     506 non-null    float64
 1   ZN       506 non-null    float64
 2   INDUS    506 non-null    float64
 3   CHAS     506 non-null    float64
 4   NOX      506 non-null    float64
 5   RM       506 non-null    float64
 6   AGE      506 non-null    float64
 7   DIS      506 non-null    float64
 8   RAD      506 non-null    float64
 9   TAX      506 non-null    float64
 10  PTRATIO  506 non-null    float64
 11  B        506 non-null    float64
 12  LSTAT    506 non-null    float64
 13  PRICE    506 non-null    float64
dtypes: float64(14)
memory usage: 55.5 KB
```

- 데이터 세트 피처는 모두 float 형
- Null값 존재X

(2) 각 컬럼이 휘귀 결과에 미치는 영향 시각화

```
fig, axs = plt.subplots(figsize=(16, 8), ncols=4, nrows=2)
lm_features = ['RM', 'ZN', "INDUS", 'NOX', 'AGE', 'PTRATIO', 'LSTAT', 'RAD']
for i, feature in enumerate(lm_features):
  row = int(i/4)
  col = i%4

  sns.regplot(x=feature, y ='PRICE', data=bostonDF, ax=axs[row][col])
```



- `enumerate()` : 인덱스 번호와 실제 값을 동시에 가져오는 함수
- RM(방 개수)은 양 방향의 선형성이 가장 큼
- LSTAT(하위 계층의 비율)는 음 방향의 선형성이 가장 큼

(3) 회귀 모델 만들기

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

y_target = bostonDF['PRICE']
X_data = bostonDF.drop(['PRICE'], axis=1, inplace=False)

X_train, X_test, y_train, y_test = train_test_split(X_data, y_target, test_size=0.3,
                                                    random_state=156)

lr = LinearRegression()
lr.fit(X_train, y_train)

y_preds = lr.predict(X_test)
mse = mean_squared_error(y_test, y_preds)
rmse = np.sqrt(mse)

print('MSE: {:.3f}, RMSE: {:.3f}'.format(mse, rmse))
print('Variance score: {:.3f}'.format(r2_score(y_test, y_preds)))
```

```
MSE: 17.297, RMSE: 4.159
Variance score: 0.757
```

(4) 회귀 계수 확인

```
print('절편 값:', lr.intercept_)
print('독립 변수의 회귀 계수:', np.round(lr.coef_, 1))
```

```
절편 값: 40.995595172164826
독립 변수의 회귀 계수: [ -0.1   0.1   0.    3.  -19.8   3.4   0.   -1.7   0.4  -0.   -0.9   0.
  -0.6]
```

```
coeff = pd.Series(data=np.round(lr.coef_, 1), index=X_data.columns)
coeff.sort_values(ascending=False)
```

	0
RM	3.4
CHAS	3.0
RAD	0.4
ZN	0.1
INDUS	0.0
B	0.0
TAX	-0.0
AGE	0.0
CRIM	-0.1
LSTAT	-0.6
PTRATIO	-0.9
DIS	-1.7
NOX	-19.8

dtype: float64

- RM이 양의 값으로 회귀 계수가 가장 큼
- NOX 피처의 회귀 계수 크기가 너무 큼 -> 최적화하면서 확인

(5) 교차 검증으로 성능 평가하기

```
from sklearn.model_selection import cross_val_score

y_target = bostonDF['PRICE']
X_data = bostonDF.drop(['PRICE'], axis=1, inplace=False)
lr = LinearRegression()

neg_mse_scores = cross_val_score(lr, X_data, y_target, scoring='neg_mean_squared_error', cv=5)
rmse_scores = np.sqrt(-1 * neg_mse_scores)
avg_rmse = np.mean(rmse_scores)

print('5 folds의 개별 Negative MSE scores:', np.round(neg_mse_scores, 2))
print('5 folds의 개별 RMSE sores:', np.round(rmse_scores, 2))
print('5 folds의 평균 RMSE: {:.3f}'.format(avg_rmse))
```

```
5 folds의 개별 Negative MSE scores: [-12.46 -26.05 -33.07 -80.76 -33.31]
5 folds의 개별 RMSE sores: [3.53 5.1  5.75 8.99 5.77]
5 folds의 평균 RMSE: 5.829
```

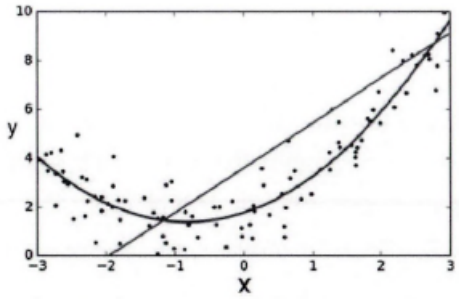
- 사이킷런의 지표 평가 기준은 높은 지표 값일수록 좋은 모델이지만 MSE값은 낮을수록 좋음

  -> `scoring='neg_mean_squared_error` 로 호출하면 모델에서 계산된 MSE 값에 -1을 곱해서 반환
- 5개 폴드 세트에 대해 교차 검증 수행 결과: 평균 RMSE가 약 5.829

05. 다항 회귀와 과(대)적합/과소적합 이해

- **다항화귀**: 2차, 3차 방정식과 같은 다항식으로 표현되는 회귀

- 비선형 회귀X, 선형 회귀O



〈 주어진 데이터 세트에서 다항 회귀가 더 효과적임 〉

다항 회귀 사이킷런

- 사이킷런은 다항 회귀를 위한 클래스를 명시적으로 제공X
- 비선형 함수를 선형 모델에 적용시키는 방법을 사용해 구현
- `PolynomialFeatures` 클래스를 통해 피처를 다항# 단항식을 2차 다항식으로 변환
- `model.intercept_` : 절편
  `model.coef_` : 독립변수의 회귀계수

단항식을 2차 다항식으로 변환

```
from sklearn.preprocessing import PolynomialFeatures
import numpy as np

X = np.arange(4).reshape(2, 2)
print('일차 단항식 계수 피처:₩n', X)

poly = PolynomialFeatures(degree=2)
poly.fit(X)
poly_ftr = poly.transform(X)
print('변환된 2차 다항식 계수 피처:₩n', poly_ftr)
```

```
일차 단항식 계수 피처:
 [[0 1]
 [2 3]]
변환된 2차 다항식 계수 피처:
 [[1. 0. 1. 0. 0. 1.]
 [1. 2. 3. 4. 6. 9.]]
```

- 기존 $[x_1, x_2]$를 2차 다항 특성 $[1, x_1, x_2, x_1^2, x_1x_2, x_2^2]$로 변환

단항식을 3차 다항식으로 변환

(1). 삼차 다항식 결정값 구하기

```
def polynomial_func(X):
  y = 1 + 2*X[:, 0] + 3*X[:, 0]**2 + 4*X[:, 1]**3
  return y

X = np.arange(4).reshape(2, 2)
print('일차 단항식 계수 feature: ₩n', X)
y = polynomial_func(X)
print('삼차 다항식 결정값: ₩n', y)
```

```
일차 단항식 계수 feature:
 [[0 1]
 [2 3]]
삼차 다항식 결정값:
 [  5 125]
```

- 회귀 결정 함수식을 임의로 $y = 1 + 2x_1 + 3x_1^2 + 4x_2^3$ 로 설정

(2). 3차 다항식으로 변환

```
poly_ftr = PolynomialFeatures(degree=3).fit_transform(X)
print('3차 다항식 계수 feature: ₩n', poly_ftr)
```

```
3차 다항식 계수 feature:
 [[ 1.  0.  1.  0.  0.  1.  0.  0.  0.  1.]
 [ 1.  2.  3.  4.  6.  9.  8. 12. 18. 27.]]
```

- 기존 $[x_1, x_2]$를 3차 다항 특성 $[1, x_1, x_2, x_1^2, x_1x_2, x_2^2, x_1^3, x_1^2x_2, x_1x_2^2, x_2^3]$ 로 변환

(3). 선형 회귀에 적용

```
model = LinearRegression()
model.fit(poly_ftr, y)
print('Polynomial 회귀 계수₩n', np.round(model.coef_, 2))
print('Polynomial 회귀 Shape:', model.coef_.shape)
```

```
Polynomial 회귀 계수
 [0.   0.18 0.18 0.36 0.54 0.72 0.72 1.08 1.62 2.34]
Polynomial 회귀 Shape: (10,)
```

사이킷런의 Pipeline 객체를 이용

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
import numpy as np

def polynomial_func(X):
  y = 1 + 2*X[:, 0] + 3*X[:, 0]**2 + 4*X[:, 1]**3
  return y

model = Pipeline([('poly', PolynomialFeatures(degree=3)),
                  ('linear', LinearRegression())])
X = np.arange(4).reshape(2, 2)
y = polynomial_func(X)

model = model.fit(X, y)

print('Polynomial 회귀 계수₩n', np.round(model.named_steps['linear'].coef_, 2))
```

```
Polynomial 회귀 계수
 [0.   0.18 0.18 0.36 0.54 0.72 0.72 1.08 1.62 2.34]
```

- Polynomial Feature 변환과 Linear Regression을 연결하여 한 번에 다항 회귀를 구현
- `named_steps['linear']` : Pipeline 안의 선형회귀 모델을 가져옴

다항 회귀를 이용한 과소적합 및 과적합 이해

- 다항식의 차수가 높아질수록 매우 복잡한 피처 간의 관계까지 모델링 가능
- 하지만 다항 회귀의 차수를 높일 수록 과적합 문제↑

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score
%matplotlib inline

# 임의의 값으로 구성된 X값에 대해 코사인 변환 값을 반환
def true_fun(X):
  return np.cos(1.5 * np.pi * X)

# X는 0부터 1까지 30개의 임의의 값을 순서대로 샘플링한 데이터
np.random.seed(0)
n_samples = 30
X = np.sort(np.random.rand(n_samples))

# y  값은 코사인 기반의 true_fun()에서 약간의 노이즈 변동 값을 더한 값
y = true_fun(X) + np.random.randn(n_samples) * 0.1
```

- 코사인 함수로 실제값을 생성하고, 여기에 랜덤 노이즈를 추가해 회귀분석에 사용할 30개의 가상 데이터(X, y)를 만드는 코드

```
plt.figure(figsize=(14, 5))
degrees = [1, 4, 15]

for i in range(len(degrees)):
  ax = plt.subplot(1, len(degrees), i+1)
  plt.setp(ax, xticks=(), yticks=())

  polynomial_features = PolynomialFeatures(degree=degrees[i], include_bias=False)
  linear_regression = LinearRegression()
  pipeline = Pipeline([('polynomial_features', polynomial_features),
                       ('linear_regression', linear_regression)])
  pipeline.fit(X.reshape(-1, 1), y)

  # 교차 검증
  scores = cross_val_score(pipeline, X.reshape(-1, 1), y, scoring='neg_mean_squared_error', cv=10)
  # 회귀 계수 추출
  coefficients = pipeline.named_steps['linear_regression'].coef_
  print('₩nDegree {} 회귀 계수는 {} 입니다.'.format(degrees[i], np.round(coefficients, 2)))
  print('Degree {} MSE는 {} 입니다.'.format(degrees[i], -1*np.mean(scores)))

  X_test = np.linspace(0, 1, 100)
  # 예측값 곡선
  plt.plot(X_test, pipeline.predict(X_test[:, np.newaxis]), label='Model')
  # 실제 값 곡선
  plt.plot(X_test, true_fun(X_test), '--', label='True function')
  plt.scatter(X, y, edgecolor='b', s=20, label='Samples')

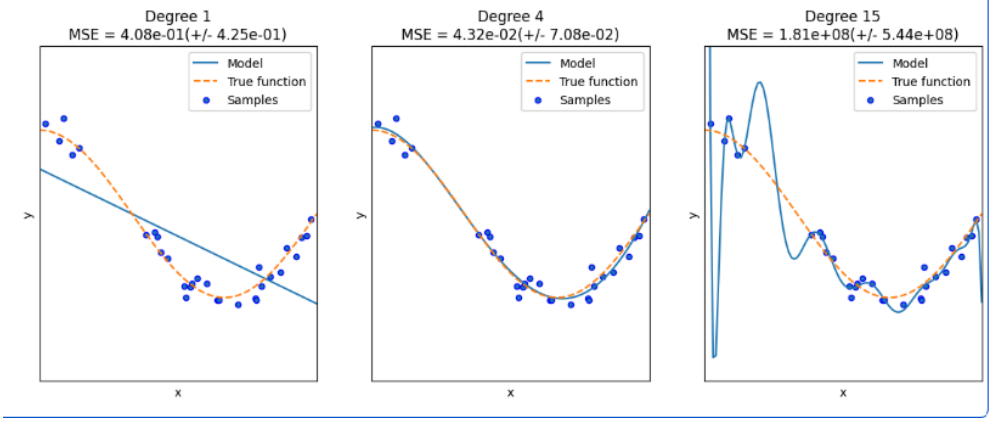
  plt.xlabel('x'); plt.ylabel('y'); plt.xlim((0, 1)); plt.ylim((-2, 2)); plt.legend(loc='best')
  plt.title('Degree {}₩nMSE = {:.2e}(+/- {:.2e})'.format(degrees[i], -scores.mean(), scores.std()))

plt.show()
```

```
Degree 1 회귀 계수는 [-1.61] 입니다.
Degree 1 MSE는 0.4077289625098685 입니다.

Degree 4 회귀 계수는 [  0.47 -17.79  23.59  -7.26] 입니다.
Degree 4 MSE는 0.04320874987232064 입니다.

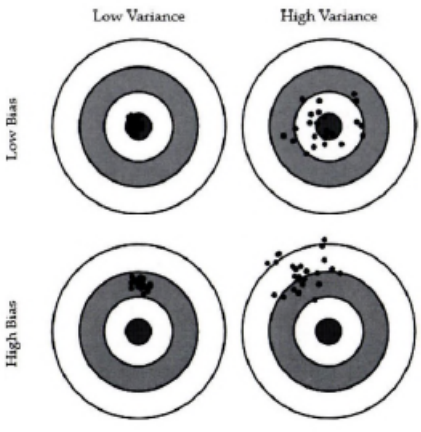
Degree 15 회귀 계수는 [-2.98293000e+03  1.03899390e+05 -1.87416123e+06  2.03716219e+07
 -1.44873283e+08  7.09315363e+08 -2.47065792e+09  6.24561050e+09
 -1.15676510e+10  1.56894936e+10 -1.54006023e+10  1.06457264e+10
 -4.91377530e+09  1.35919645e+09 -1.70380786e+08] 입니다.
Degree 15 MSE는 181238256.56423894 입니다.
```



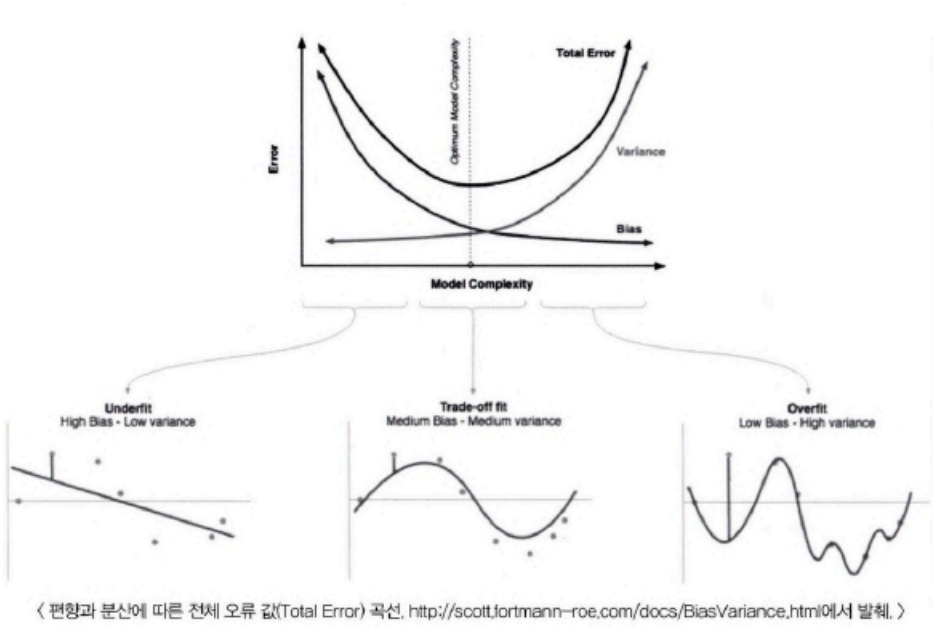
- 맨 왼쪽의 Degree 1 예측 곡선: 단순 선형 회귀로 과소적합 모델, MSE=0.41
- 가운데 Degree 4 예측 곡선: 실제 데이터 세트와 유사, 테스트 데이터를 잘 예측한 모델, MSE=0.04
- 맨 오른쪽 Degree 15 예측 곡선: 과적합이 심한 모델, MSE가 매우 큼

편향-분산 트레이드오프

- 매우 단순화된 모델 -> High Bias
- 매우 복잡한 모델 -> High Variance



〈 편향과 분산의 고/저에 따른 표현.
http://scott.fortmann-roe.com/docs/BiasVariance.html에서 발췌 〉



〈 편향과 분산에 따른 전체 오류 값(Total Error) 곡선, http://scott.fortmann-roe.com/docs/BiasVariance.html에서 발췌. 〉

- 편향이 높으면 분산은 낮아지고, 분산이 높으면 편향이 낮아짐
  -> 평향과 분산이 서로 트레이드오프를 이루면서 오류 Cost 값이 최대로 낮아지는 모델을 구축하는 것이 목표

06. 규제 선형 모델 - 릿지, 라쏘, 엘라스틱넷

규제 선형 모델의 개요

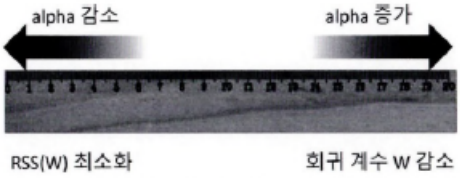


- RSS 최소화 방법과 과적합을 방지하기 위해 회귀 계수 값이 커지지 않도록 하는 방법이 서로 균형을 이루어야 함

- **비용 함수 목표**: $Min(RSS(W) + \alpha \cdot \|W\|_2^2)$

- alpha: 학습 데이터 적합 정도와 회귀 계수 값의 크기 제어를 수행하는 튜닝 파라미터
- alpha(α)가 작을수록 →회귀계수(W)를 자유롭게 조정할 수 있어 과적합 위험이 커짐
  alpha(α)가 클수록 → 회귀계수(W)를 0에 가깝게 줄여 과적합을 방지하지만, 너무 크면 과소적합이 발생할 수 있음

- alpha = 0인 경우는 W가 커도 alpha * ||W||2가 0이 되어 비용 함수는 Min(RSS(W))
- alpha = 무한대인 경우 alpha * ||W||2도 무한대가 되므로 비용 함수는 W를 0에 가깝게 최소화 해야 함.



〈alpha 튜닝 파라미터를 통한 RSS 최소화와 회귀 계수 크기 감소의 균형 조정〉

- 즉, α를 0에서부터 지속적으로 값을 증가시키면 회귀 계수 값의 크기를 감소시킬 수 있음

- **규제**: α값으로 페널티를 부여해 회귀 계수 값의 크기를 감소시켜 과적합을 개선하는 방식

- 릿지와 라쏘 회귀

릿지 회귀

- **릿지 회귀**: L2 규제를 적용한 회귀

- 사이킷런은 `Ridge` 클래스를 통해 릿지 회귀를 구현

- **L2 규제**: $\alpha \cdot \|W\|_2^2$와 같이 W의 제곱에 대해 패널티를 부여하는 방식

(1) Ridge 회귀 구현

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import cross_val_score

ridge = Ridge(alpha = 10)
neg_mse_scores = cross_val_score(ridge, X_data, y_target, scoring='neg_mean_squared_error', cv=5)
rmse_scores = np.sqrt(-1*neg_mse_scores)
avg_rmse = np.mean(rmse_scores)

print('5 folds의 개별 Negative MSE scores:', np.round(neg_mse_scores, 3))
print('5 folds의 개별 RMSE scores:', np.round(rmse_scores, 3))
print('5 folds의 평균 RMSE: {:.3f}'.format(avg_rmse))
```

```
5 folds의 개별 Negative MSE scores: [-11.422 -24.294 -28.144 -74.599 -28.517]
5 folds의 개별 RMSE scores: [3.38  4.929 5.305 8.637 5.34 ]
5 folds의 평균 RMSE: 5.518
```

- 평균 RMSE 5.829 -> 5.518로 성능 개선

(2) alpha 값을 변화시키면서 릿지 회귀 구현

```
alphas = [0, 0.1, 1, 10, 100]

for alpha in alphas:
  ridge = Ridge(alpha=alpha)

  neg_mse_scores = cross_val_score(ridge, X_data, y_target, scoring='neg_mean_squared_error', cv=5)
  avg_rmse = np.mean(np.sqrt(-1*neg_mse_scores))
  print('alpha {} 일 때 5 folds의 평균 RMSE: {:.3f}'.format(alpha, avg_rmse))
```

```
alpha 0 일 때 5 folds의 평균 RMSE: 5.829
alpha 0.1 일 때 5 folds의 평균 RMSE: 5.788
alpha 1 일 때 5 folds의 평균 RMSE: 5.653
alpha 10 일 때 5 folds의 평균 RMSE: 5.518
alpha 100 일 때 5 folds의 평균 RMSE: 5.330
```

- alpha가 100일 때 평균 RMS가 평균 5.330으로 가장 좋게 나타남

(2)-2. alpha 값의 변화에 따른 피처 회귀 계수 값 시각화

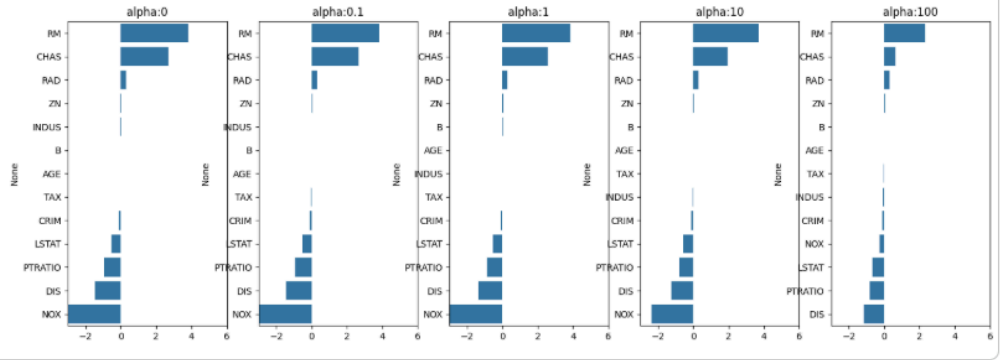
```
fig, axs = plt.subplots(figsize=(18, 6), nrows=1, ncols=5)
coeff_df = pd.DataFrame()

for pos, alpha in enumerate(alphas):
  ridge = Ridge(alpha=alpha)
  ridge.fit(X_data, y_target)

  coeff = pd.Series(data=ridge.coef_, index=X_data.columns)
  colname = 'alpha:'+str(alpha)
  coeff_df[colname] = coeff

  coeff = coeff.sort_values(ascending=False)
  axs[pos].set_title(colname)
  axs[pos].set_xlim(-3, 6)
  sns.barplot(x=coeff.values, y=coeff.index, ax=axs[pos])

plt.show()
```



- alpha 값을 계속 증가시킬수록 회귀 계수 값은 지속적으로 작아짐(특히 NOX 피처)

(2)-3. alpha 값의 변화에 따른 릿지 회귀 계수 값

```
ridge_alphas = [0, 0.1, 1, 10, 100]
sort_column = 'alpha:'+str(ridge_alphas[0])
coeff_df.sort_values(by=sort_column, ascending=False)
```

	alpha:0	alpha:0.1	alpha:1	alpha:10	alpha:100
RM	3.809865	3.818233	3.854000	3.702272	2.334536
CHAS	2.686734	2.670019	2.552393	1.952021	0.638335
RAD	0.306049	0.303515	0.290142	0.279596	0.315358
ZN	0.046420	0.046572	0.047443	0.049579	0.054496
INDUS	0.020559	0.015999	-0.008805	-0.042962	-0.052826
B	0.009312	0.009368	0.009673	0.010037	0.009393
AGE	0.000692	-0.000269	-0.005415	-0.010707	0.001212
TAX	-0.012335	-0.012421	-0.012912	-0.013993	-0.015856
CRIM	-0.108011	-0.107474	-0.104595	-0.101435	-0.102202
LSTAT	-0.524758	-0.525966	-0.533343	-0.559366	-0.660764
PTRATIO	-0.952747	-0.940759	-0.876074	-0.797945	-0.829218
DIS	-1.475567	-1.459626	-1.372654	-1.248808	-1.153390
NOX	-17.766611	-16.684645	-10.777015	-2.371619	-0.262847

- 회귀 계수가 지속적으로 작아지고 있지만 회귀 계수를 0으로 만들지는 않음

라쏘 회귀

- **라쏘 회귀**: L1 규제를 선형 회귀에 적용한 것

- 목표: $RSS(W) + \alpha \cdot \|W\|_1$를 최소화하는 W 찾기
- 사이킷런 `Lasso` 클래스를 통해 라쏘 회귀 구현

- **L1 규제**: $\alpha \cdot \|W\|_1$와 같이 W의 절댓값에 대해 페널티를 부여

- 영향력이 크지 않은 회귀 계수 값을 0으로 변환
- 적절한 피처만 회귀에 포함시키는 피처 선택의 특성을 가지고 있음

(1) alpha 값에 따른 폴드 평균 RMSE를 출력하고 회귀 계수값들을 반환

```
from sklearn.linear_model import Lasso, ElasticNet

def get_linear_reg_eval(model_name, params=None, X_data_n=None, y_target_n=None,
                        verbose=True, return_coeff=True):
  coeff_df = pd.DataFrame()
  if verbose: print('#######', model_name, '#######')
  for param in params:
    if model_name =='Ridge': model = Ridge(alpha=param)
    elif model_name == 'Lasso': model=Lasso(alpha=param)
    elif model_name == 'ElasticNet': model = ElasticNet(alpha=param, l1_ratio=0.7)
    neg_mse_scores = cross_val_score(model, X_data_n, y_target_n,
                                     scoring='neg_mean_squared_error', cv=5)
    avg_rmse = np.mean(np.sqrt(-1*neg_mse_scores))
    print('alpha {}일 때 5 폴트 세트의 평균 RMSE: {:.3f}'.format(param, avg_rmse))

    model.fit(X_data_n, y_target_n)
    if return_coeff:
      coeff = pd.Series(data=model.coef_, index=X_data_n.columns)
      colname='alpha:'+str(param)
      coeff_df[colname] = coeff

  return coeff_df
```

```
lasso_alphas = [0.07, 0.1, 0.5, 1, 3]
coeff_lasso_df = get_linear_reg_eval('Lasso', params=lasso_alphas,
                                     X_data_n=X_data, y_target_n=y_target)
```

```
####### Lasso #######
alpha 0.07일 때 5 폴트 세트의 평균 RMSE: 5.612
alpha 0.1일 때 5 폴트 세트의 평균 RMSE: 5.615
alpha 0.5일 때 5 폴트 세트의 평균 RMSE: 5.669
alpha 1일 때 5 폴트 세트의 평균 RMSE: 5.776
alpha 3일 때 5 폴트 세트의 평균 RMSE: 6.189
```

- alpha가 0.07일 때 5.612로 가장 좋은 평균 RMSE를 보여줌
- 5.829 -> 5.612로 향상

(1)-2. alpha 값에 따른 피처별 회귀 계수

```
sort_column = 'alpha:'+str(lasso_alphas[0])
coeff_lasso_df.sort_values(by=sort_column, ascending=False)
```

	alpha:0.07	alpha:0.1	alpha:0.5	alpha:1	alpha:3
RM	3.789725	3.703202	2.498212	0.949811	0.000000
CHAS	1.434343	0.955190	0.000000	0.000000	0.000000
RAD	0.270936	0.274707	0.277451	0.264206	0.061864
ZN	0.049059	0.049211	0.049544	0.049165	0.037231
B	0.010248	0.010249	0.009469	0.008247	0.006510
NOX	-0.000000	-0.000000	-0.000000	-0.000000	0.000000
AGE	-0.011706	-0.010037	0.003604	0.020910	0.042495
TAX	-0.014290	-0.014570	-0.015442	-0.015212	-0.008602
INDUS	-0.042120	-0.036619	-0.005253	-0.000000	-0.000000
CRIM	-0.098193	-0.097894	-0.083289	-0.063437	-0.000000
LSTAT	-0.560431	-0.568769	-0.656290	-0.761115	-0.807679
PTRATIO	-0.765107	-0.770654	-0.758752	-0.722966	-0.265072
DIS	-1.176583	-1.160538	-0.936605	-0.668790	-0.000000

- alpha의 크기가 증가함에 따라 일부 피처의 회귀 계수가 아예 0으로 바뀜

엘라스틱넷 회귀

- **엘라스틱넷 회귀**: L2규제와 L1 규제를 결합한 회귀

- 목표: $RSS(W) + alpha2 \cdot \|W\|_2^2 + alpha1 \cdot \|W\|_1$ 식을 최소화하는 W 찾기
- 라쏘 회귀: 서로 상관관계가 높은 피처들의 경우에 이들 중에서 중요 피처만을 셀렉션하고 다른 피처들은 모두 회귀 계수를 0으로 만드는 성향이 강함
  -> alpha값에 따라 회귀 계수의 값이 급격하게 변동할 수 있음 -> L2 규제 추가
- 단점: 수행시간↑
- 사이킷런은 `ElasticNet` 클래스를 통해 엘라스틱넷 회귀를 구현
- 엘라스틱넷의 규제: (a*L1* + b*L2*)로 정의 <- a는 L1 규제의 alpha값, b는 L2 규제의 alpha 값
- 따라서 ElasticNet 클래스의 `alpha` 파라미터 값은 a+b
- ElasticNet 클래스의 `l1_ratio` 파라미터 값은 a/(a+b)

엘라스틱넷 alpha값에 따른 RMSE와 각 피처의 회귀 계수 출력

```
# l1_ratio는 0.7로 고정
elastic_alphas = [0.07, 0.1, 0.5, 1, 3]
coeff_elastic_df = get_linear_reg_eval('ElasticNet', params=elastic_alphas,
                                       X_data_n = X_data, y_target_n=y_target)
```

```
####### ElasticNet #######
alpha 0.07일 때 5 폴트 세트의 평균 RMSE: 5.542
alpha 0.1일 때 5 폴트 세트의 평균 RMSE: 5.526
alpha 0.5일 때 5 폴트 세트의 평균 RMSE: 5.467
alpha 1일 때 5 폴트 세트의 평균 RMSE: 5.597
alpha 3일 때 5 폴트 세트의 평균 RMSE: 6.068
```

```
sort_column = 'alpha:'+str(elastic_alphas[0])
coeff_elastic_df.sort_values(by=sort_column, ascending=False)
```

	alpha:0.07	alpha:0.1	alpha:0.5	alpha:1	alpha:3
RM	3.574162	3.414154	1.918419	0.938789	0.000000
CHAS	1.330724	0.979706	0.000000	0.000000	0.000000
RAD	0.278880	0.283443	0.300761	0.289299	0.146846
ZN	0.050107	0.050617	0.052878	0.052136	0.038268
B	0.010122	0.010067	0.009114	0.008320	0.007020
AGE	-0.010116	-0.008276	0.007760	0.020348	0.043446
TAX	-0.014522	-0.014814	-0.016046	-0.016218	-0.011417
INDUS	-0.044855	-0.042719	-0.023252	-0.000000	-0.000000
CRIM	-0.099468	-0.099213	-0.089070	-0.073577	-0.019058
NOX	-0.175072	-0.000000	-0.000000	-0.000000	-0.000000
LSTAT	-0.574822	-0.587702	-0.693861	-0.760457	-0.800368
PTRATIO	-0.779498	-0.784725	-0.790969	-0.738672	-0.423065
DIS	-1.189438	-1.173647	-0.975902	-0.725174	-0.031208

- alpha가 0.5일 때 RMSE가 5.467로 가장 좋은 예측 성능을 보임

- alpha값에 따른 피처들의 회귀 계수들 값이 라쏘보다는 상대적으로 0이 되는 값이 적음

선형 회귀 모델을 위한 데이터 변환

- 선형회귀는 데이터가 정규분포에 가까울수록 유리한 경우가 있으므로,
  분포가 심하게 치우쳐 있다면 스케일링/정규화 작업을 수행하는 것이 일반적

(1) 사이킷런을 이용해 피처 데이터 세트에 적용하는 변환 작업 순서

1. StandardScaler 클래스를 이용해 평균이 0, 분산이 1인 표준 정규 분포를 가진 데이터 세트로 변환하거나 MinMaxScaler 클래스를 이용해 최솟값이 0이고 최댓값이 1인 값으로 정규화를 수행합니다.
2. 스케일링/정규화를 수행한 데이터 세트에 다시 다항 특성을 적용하여 변환하는 방법입니다. 보통 1번 방법을 통해 예측 성능에 향상이 없을 경우 이와 같은 방법을 적용합니다.
3. 원래 값에 log 함수를 적용하면 보다 정규 분포에 가까운 형태로 값이 분포됩니다. 이러한 변환을 로그 변환(Log Transformation)이라고 부릅니다. 로그 변환은 매우 유용한 변환이며, 실제로 선형 회귀에서는 앞에서 소개한 1, 2번 방법보다 로그 변환이 훨씬 많이 사용되는 변환 방법입니다. 왜냐하면 1번 방법의 경우 예측 성능 향상을 크게 기대하기 어려운 경우가 많으며 2번 방법의 경우 피처의 개수가 매우 많을 경우에는 다항 변환으로 생성되는 피처의 개수가 기하급수로 늘어나서 과적합의 이슈가 발생할 수 있기 때문입니다.

(2)-1. 변환 방법을 결정하여 스케일링하는 함수

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, PolynomialFeatures

def get_scaled_data(method='None', p_degree=None, input_data=None):
  if method == 'Standard':
    scaled_data = StandardScaler().fit_transform(input_data)
  elif method == 'MinMax':
    scaled_data = MinMaxScaler().fit_transform(input_data)
  elif method == 'Log':
    scaled_data = np.log1p(input_data)
  else:
    scaled_data = input_data

  if p_degree != None:
    scaled_data = PolynomialFeatures(degree=p_degree,
                                     include_bias=False).fit_transform(scaled_data)

  return scaled_data
```

(2)-2. 데이터 변환 방법과 alpha값에 따른 릿지 회귀 모델의 RMSE을 비교

```
alphas = [0.1, 1, 10, 100]

scale_methods = [(None, None), ('Standard', None), ('Standard', 2),
                 ('MinMax', None), ('MinMax', 2), ('Log', None)]
for scale_method in scale_methods:
  X_data_scaled = get_scaled_data(method=scale_method[0], p_degree=scale_method[1],
                                  input_data=X_data)
  print('₩n## 변환 유형: {}, Polynomial Degree: {}'.format(scale_method[0], scale_method[1]))
  get_linear_reg_eval('Ridge', params=alphas, X_data_n = X_data_scaled,
                      y_target_n=y_target, verbose=False, return_coeff=False)
```

```
## 변환 유형: None, Polynomial Degree: None
alpha 0.1일 때 5 폴트 세트의 평균 RMSE: 5.788
alpha 1일 때 5 폴트 세트의 평균 RMSE: 5.653
alpha 10일 때 5 폴트 세트의 평균 RMSE: 5.518
alpha 100일 때 5 폴트 세트의 평균 RMSE: 5.330

## 변환 유형: Standard, Polynomial Degree: None
alpha 0.1일 때 5 폴트 세트의 평균 RMSE: 5.826
alpha 1일 때 5 폴트 세트의 평균 RMSE: 5.803
alpha 10일 때 5 폴트 세트의 평균 RMSE: 5.637
alpha 100일 때 5 폴트 세트의 평균 RMSE: 5.421

## 변환 유형: Standard, Polynomial Degree: 2
alpha 0.1일 때 5 폴트 세트의 평균 RMSE: 8.827
alpha 1일 때 5 폴트 세트의 평균 RMSE: 6.871
alpha 10일 때 5 폴트 세트의 평균 RMSE: 5.485
alpha 100일 때 5 폴트 세트의 평균 RMSE: 4.634

## 변환 유형: MinMax, Polynomial Degree: None
alpha 0.1일 때 5 폴트 세트의 평균 RMSE: 5.764
alpha 1일 때 5 폴트 세트의 평균 RMSE: 5.465
alpha 10일 때 5 폴트 세트의 평균 RMSE: 5.754
alpha 100일 때 5 폴트 세트의 평균 RMSE: 7.635

## 변환 유형: MinMax, Polynomial Degree: 2
alpha 0.1일 때 5 폴트 세트의 평균 RMSE: 5.298
alpha 1일 때 5 폴트 세트의 평균 RMSE: 4.323
alpha 10일 때 5 폴트 세트의 평균 RMSE: 5.185
alpha 100일 때 5 폴트 세트의 평균 RMSE: 6.538

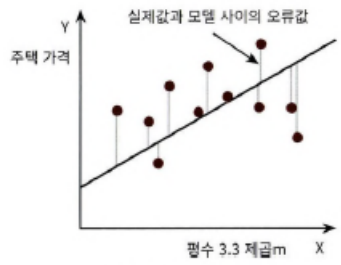
## 변환 유형: Log, Polynomial Degree: None
alpha 0.1일 때 5 폴트 세트의 평균 RMSE: 4.770
alpha 1일 때 5 폴트 세트의 평균 RMSE: 4.676
alpha 10일 때 5 폴트 세트의 평균 RMSE: 4.836
alpha 100일 때 5 폴트 세트의 평균 RMSE: 6.241
```

- 표준 정규 분포와 최솟값/최댓값 정규화로 피처 데이터 세트를 변경해도 성능상의 개선은 없음
- 표준 정규 분포로 일차 변환 후 2차 다항식 변환을 했을 때 alpha=100에서 4.634로 성능 개선
- 최솟값/최댓값 정규화로 일차 변환 후 2차 다항식 변환 했을 때 alpha=1에서 4.323으로 성능 개선

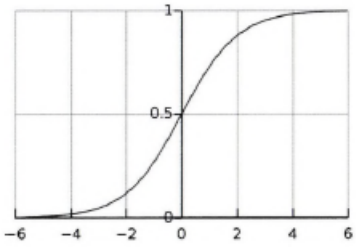
07. 로지스틱 회귀

-**로지스틱 회귀**: 선형 회귀 방식을 분류에 적용한 알고리즘

- 선형 함수의 회귀 최적선을 찾는 것이 아니라 **시그모이드**(sigmoid) 함수 최적선을 찾고 이 시그모이드 함수의 반환 값을 확률로 간주해 확률에 따라 분류



〈 선형 회귀의 선형 함수 〉



〈 로지스틱 회귀의 시그모이드 함수 〉

시그모이드 함수: $y = \frac{1}{1+e^{-z}}$

- x 값이 아무리 커지거나 작아져도 y 값은 항상 0과 1 사이 값을 반환
- 사이킷런은 `LogisticRegression` 클래스를 제공
- `LogisticRegression` 클래스에서 `solver` 파라미터의 'lbfgs', 'liblinear,'newton-cg', 'sag', 'saga' 값을 적용해서 최적화를 선택

- lbfgs: 사이킷런 버전 0.22부터 solver의 기본 설정값입니다. 메모리 공간을 절약할 수 있고, CPU 코어 수가 많다면 최적화를 병렬로 수행할 수 있습니다.
- liblinear: 사이킷런 버전 0.21까지에서 solver의 기본 설정값입니다. 다차원이고 작은 데이터 세트에서 효과적으로 동작하지만 국소 최적화(Local Minimum)에 이슈가 있고, 병렬로 최적화할 수 없습니다.
- newton-cg: 좀 더 정교한 최적화를 가능하게 하지만, 대용량의 데이터에서 속도가 많이 느려집니다.
- sag: Stochastic Average Gradient로서 경사 하강법 기반의 최적화를 적용합니다. 대용량의 데이터에서 빠르게 최적화합니다.
- saga: sag와 유사한 최적화 방식이며 L1 정규화를 가능하게 해줍니다.

로지스틱 회귀로 암 여부 판단하기

(1) 전체 데이터 세트 생성

```
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression

cancer = load_breast_cancer()
```

(2) 표준 스케일링 적용 후 데이터 세트 분리

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

scaler = StandardScaler()
data_scaled = scaler.fit_transform(cancer.data)

X_train, X_test, y_train, y_test = train_test_split(data_scaled, cancer.target,
                                                    test_size=0.3, random_state=0)
```

(3) 로지스틱 회귀를 이용해 학습 및 예측 수행

```
from sklearn.metrics import accuracy_score, roc_auc_score

lr_clf = LogisticRegression()
lr_clf.fit(X_train, y_train)

lr_preds = lr_clf.predict(X_test)
lr_preds_proba = lr_clf.predict_proba(X_test)[:, 1]
```

- 기본 `solver` 값은 'lbfgs'

(4) 정확도와 ROC-AUC 값 구하기

```
print('accuracy: {:.3f}, roc_auc:{:.3f}'.format(accuracy_score(y_test, lr_preds),
                                                roc_auc_score(y_test, lr_preds_proba)))
```

```
accuracy: 0.977, roc_auc:0.995
```

- 정확도 0.977, ROC-AUC 0.955

(5) 서로 다른 solver 값으로 학습하고 성능 평가

```
solvers = ['lbfgs', 'liblinear', 'newton-cg', 'sag', 'saga']

for solver in solvers:
  lr_clf = LogisticRegression(solver=solver, max_iter=600)
  lr_clf.fit(X_train, y_train)
  lr_preds = lr_clf.predict(X_test)
  lr_preds_proba = lr_clf.predict_proba(X_test)[:, 1]

  print('solver: {}, accuracy: {:.3f}, roc_auc: {:.3f}'.format(solver,
                                                accuracy_score(y_test, lr_preds),
                                                roc_auc_score(y_test, lr_preds_proba)))
```

```
solver: lbfgs, accuracy: 0.977, roc_auc: 0.995
solver: liblinear, accuracy: 0.982, roc_auc: 0.995
solver: newton-cg, accuracy: 0.977, roc_auc: 0.995
solver: sag, accuracy: 0.982, roc_auc: 0.995
solver: saga, accuracy: 0.982, roc_auc: 0.995
```

- liblinear와 sag, saga의 경우 newton-cg에 대비하여 상대적인 성능 수치가 약가 논ㅍ지만 데이터 세트가 작아 크게 의미 있는 결과는 아님

(6) GridSearchCV를 이용해 solver, penalty, C 최적화

```
from sklearn.model_selection import GridSearchCV

params = {'solver':['liblinear', 'lbfgs'],
          'penalty':['l2', 'l1'],
          'C':[0.01, 0.1, 1, 5, 10]}

lr_clf = LogisticRegression()

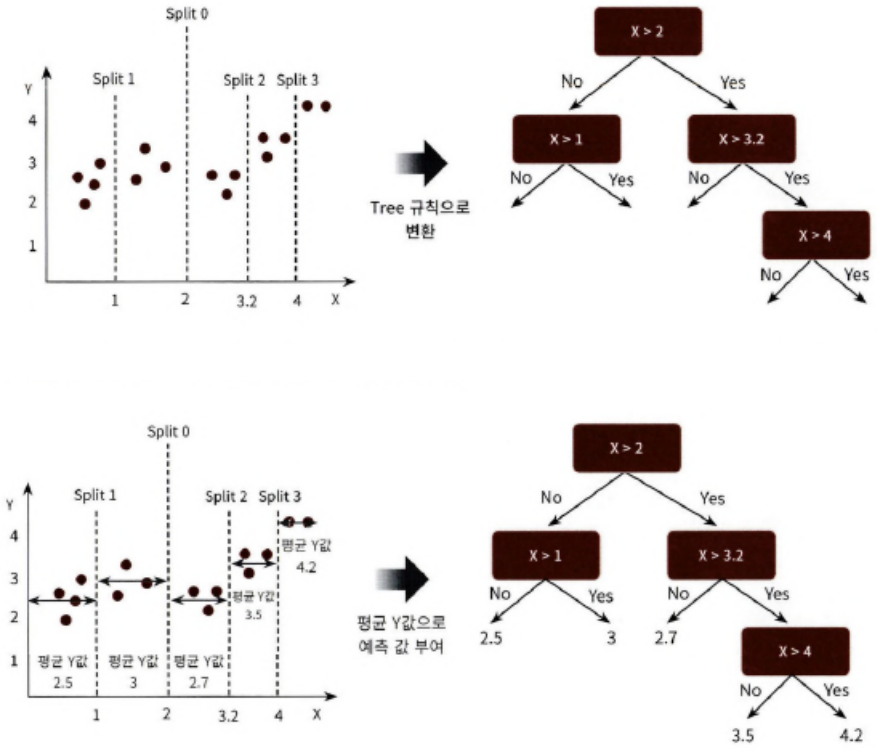
grid_clf = GridSearchCV(lr_clf, param_grid=params, scoring='accuracy', cv=3)
grid_clf.fit(data_scaled, cancer.target)
print('최적 하이퍼 파라미터: {}, 최적 평균 정확도: {:.3f}'.format(grid_clf.best_params_,
                                                grid_clf.best_score_))
```

```
최적 하이퍼 파라미터: {'C': 0.1, 'penalty': 'l2', 'solver': 'liblinear'}, 최적 평균 정확도: 0.979
```

08. 회귀 트리

- **회귀 트리**: 선형회귀와 달리 데이터를 트리 구조로 나누고, 각 리프 노드의 평균값을 이용해 예측하는 회귀 모델

- 결정 트리, 랜덤 포레스트, GBM, XGBoost, LightGBM 등은 분류뿐 아니라 회귀에도 사용 가능
- **CART** 알고리즘 기반으로 분류와 회귀 모두 수행



회귀 트리를 이용해 보스턴 주택 가격 예측 수행

(1) 랜덤 포레스트 회귀 트리

```
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestRegressor
import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore')

columns = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE',
           'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT', 'PRICE']

bostonDF = pd.read_csv('housing.csv', sep=r'\s+', header=None,
                       names=columns).astype(float)

y_target = bostonDF['PRICE']
X_data = bostonDF.drop(['PRICE'], axis=1, inplace=False)

rf = RandomForestRegressor(random_state=0, n_estimators=1000)
neg_mse_scores = cross_val_score(rf, X_data, y_target, scoring='neg_mean_squared_error', cv=5)
rmse_scores = np.sqrt(-1*neg_mse_scores)
avg_rmse = np.mean(rmse_scores)

print('5 교차 검증의 개별 Negative MSE scores:', np.round(neg_mse_scores, 2))
print('5 교차 검증의 개별 RMSE scores:', np.round(rmse_scores, 2))
print('5 교차 검증의 평균 RMSE: {:.3f}'.format(avg_rmse))
```

```
5 교차 검증의 개별 Negative MSE scores: [ -7.88 -13.14 -20.57 -46.23 -18.88]
5 교차 검증의 개별 RMSE scores: [2.81 3.63 4.54 6.8  4.34]
5 교차 검증의 평균 RMSE: 4.423
```

(2) 결정 트리, GBM, XGBoost, LightGBM 회귀를 이용

```
def get_model_cv_prediction(model, X_data, y_target):
  neg_mse_scores=cross_val_score(model, X_data, y_target, scoring='neg_mean_squared_error', cv=5)
  rmse_scores = np.sqrt(-1*neg_mse_scores)
  avg_rmse = np.mean(rmse_scores)
  print('####', model.__class__.__name__, '####')
  print('5 교차 검증의 평균 RMSE: {:.3f}'.format(avg_rmse))


from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import GradientBoostingRegressor
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor

dt_reg = DecisionTreeRegressor(random_state=0, max_depth=4)
rf_reg = RandomForestRegressor(random_state=0, n_estimators=1000)
gb_reg = GradientBoostingRegressor(random_state=0, n_estimators=1000)
xgb_reg = XGBRegressor(n_estimators=1000)
lgb_reg = LGBMRegressor(n_estimators=1000, verbose=-1)

models = [dt_reg, rf_reg, gb_reg, xgb_reg, lgb_reg]
for model in models:
  get_model_cv_prediction(model, X_data, y_target)
```

```
#### DecisionTreeRegressor ####
5 교차 검증의 평균 RMSE: 5.978
#### RandomForestRegressor ####
5 교차 검증의 평균 RMSE: 4.423
#### GradientBoostingRegressor ####
5 교차 검증의 평균 RMSE: 4.269
#### XGBRegressor ####
5 교차 검증의 평균 RMSE: 4.968
#### LGBMRegressor ####
5 교차 검증의 평균 RMSE: 4.646
```

(3) 피처별 중요도 시각화

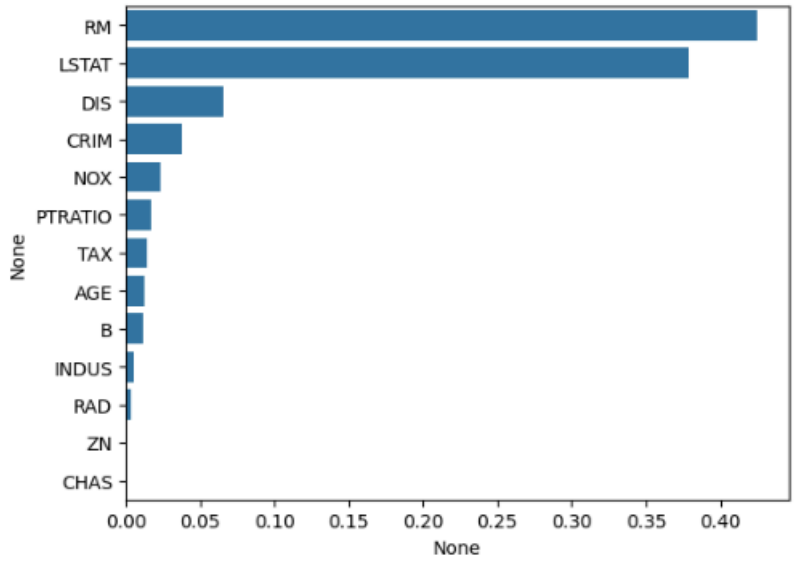
```
import seaborn as sns
%matplotlib inline

rf_reg = RandomForestRegressor(n_estimators=1000)

rf_reg.fit(X_data, y_target)

feature_series = pd.Series(data=rf_reg.feature_importances_, index=X_data.columns)
feature_series = feature_series.sort_values(ascending=False)
sns.barplot(x=feature_series, y=feature_series.index)
```

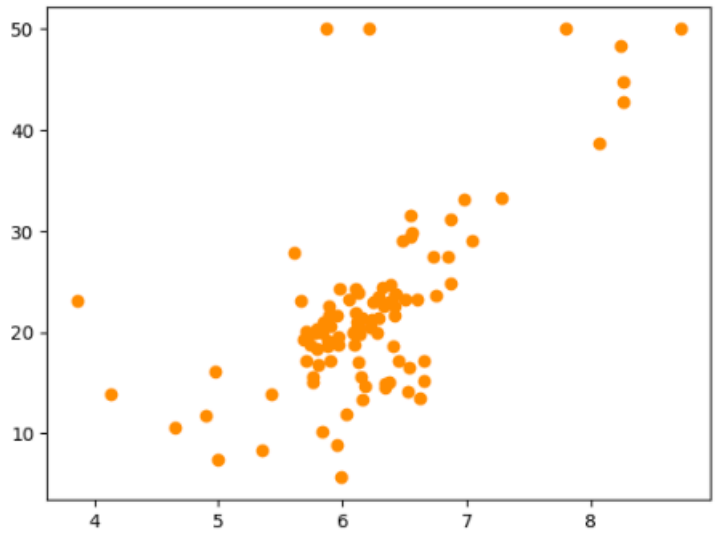
```
<Axes: xlabel='None', ylabel='None'>
```



(4) 데이터 세트 100개 샘플링 후 RM과 PRICE의 관계 산점도

```
bostonDF_sample = bostonDF[['RM', 'PRICE']]
bostonDF_sample = bostonDF_sample.sample(n=100, random_state=0)
print(bostonDF_sample.shape)
plt.figure()
plt.scatter(bostonDF_sample.RM, bostonDF_sample.PRICE, c='darkorange')
```

```
(100, 2)
<matplotlib.collections.PathCollection at 0x7b298d16cf50>
```



(5) 선형회귀와 회귀트리 학습 및 예측

```
import numpy as np
from sklearn.linear_model import LinearRegression

lr_reg = LinearRegression()
rf_reg2 = DecisionTreeRegressor(max_depth=2)
rf_reg7 = DecisionTreeRegressor(max_depth=7)

X_test = np.arange(4.5, 8.5, 0.04).reshape(-1, 1)

X_feature = bostonDF_sample['RM'].values.reshape(-1, 1)
y_target = bostonDF_sample['PRICE'].values.reshape(-1, 1)

lr_reg.fit(X_feature, y_target)
rf_reg2.fit(X_feature, y_target)
rf_reg7.fit(X_feature, y_target)

pred_lr = lr_reg.predict(X_test)
pred_rf2 = rf_reg2.predict(X_test)
pred_rf7 = rf_reg7.predict(X_test)
```

(6) 회귀선 그리기

```
fig, (ax1, ax2, ax3) = plt.subplots(figsize=(14, 4), ncols=3)

ax1.set_title('Linear Regression')
ax1.scatter(bostonDF_sample.RM, bostonDF_sample.PRICE, c="darkorange")
ax1.plot(X_test, pred_lr, label="linear", linewidth=2)

ax2.set_title('Decision Tree Regression: \n max_depth=2')
ax2.scatter(bostonDF_sample.RM, bostonDF_sample.PRICE, c="darkorange")
ax2.plot(X_test, pred_rf2, label="max_depth:3", linewidth=2)

ax3.set_title('Decision Tree Regression: \n max_depth=7')
ax3.scatter(bostonDF_sample.RM, bostonDF_sample.PRICE, c="darkorange")
ax3.plot(X_test, pred_rf7, label="max_depth:7", linewidth=2)
```

```
[<matplotlib.lines.Line2D at 0x7b298d3ec7d0>]
```



- 선형 회귀는 직선으로 예측 회귀선을 표현하지만, 회귀 트리는 분할되는 데이터 지점에 따라 브랜치를 만들면서 계단 형태로 회귀선을 만듦
- `max_depth=7` 의 경우 과적합 가능성 ↑



조윤서

이전 포스트
[Euron] 머신러닝 스터디 5주차

0개의 댓글

댓글을 작성하세요

댓글 작성

관심 있을 만한 포스트

